Tadori v2.1

Frozen Implementation Corrections

Frozen specification package

July 2026

Table of Contents

1 Tadori v2.1 - Frozen Implementation Corrections

Status: Frozen and accepted
Base document: *Tadori v2: Technical Concept*
Rule: Where this document conflicts with Tadori v2, this document controls.

1.1 Locked decisions

- Working name: Tadori.
- Initial scope: TypeScript and JavaScript only.
- Primary workflow: supervising Claude Code, with Codex support through the same MCP-oriented core where practical.
- Product modes: Review mode is primary; Explore mode exists to make Review mode understandable.
- Exactly six initial MCP tools: repo_overview, find_symbol, symbol_context, find_tests, impact, and path.
- Stable 2D is the default view. 2.5D and 3D are experiments with removal criteria.
- No runtime tracing in the prototype.
- No undocumented design rationale presented as fact.
- No dependency on ATLAS.
- The product compares extracted graph snapshots; it does not claim complete runtime architecture truth.

1.2 1. Stable entities and snapshot memberships

Tadori uses stable entities plus snapshot-membership rows. It does not overwrite one mutable graph and it does not model every state as a Git commit.

Supported repository-state kinds:

- commit
- working_tree

- staged
- patch

A snapshot records its repository, kind, base commit when available, deterministic workspace hash, parent snapshot, creation time, and pin status.

1.2.1 Stable identities

Canonical identities use printable pipe separators. Before joining fields, escape backslashes and then pipes.

- File: file|<normalized path>
- Node: node|<kind>|<qualified name>
- Edge: edge|<source node key>|<relation>|<destination node key>

Canonical identities are UTF-8 encoded and SHA-256 hashed. Collision handling compares the full canonical identity; if a true collision occurs, a collision index is appended and the key is rehashed.

Node identity remains based on (kind, qualified_name). Edge identity remains based on (source key, relation, destination key).

1.2.2 Snapshot memberships

Per-snapshot rows carry state-specific data such as:

- path and content hash;
- package;
- source spans and lines;
- signature and body hash;
- edge provenance, confidence, and resolution;
- analyzer version;
- evidence locations.

This preserves stable references for layout positions, retrieval events, annotations, decisions, and task records while allowing multiple repository states to coexist.

1.3 2. Correct graph-diff language

Permitted claim:

> Tadori computes the exact set difference between two extracted graph snapshots.

Prohibited claim:

> Tadori computes the complete or exact change in the repository's real runtime architecture.

Diff classes:

- Added compiler-resolved relationship
- Removed compiler-resolved relationship
- Added likely extracted relationship
- Removed likely extracted relationship
- Added inferred relationship - verify in source
- Analyzer no longer infers this relationship
- Unresolved target
- Relationship resolution strengthened
- Relationship resolution weakened
- Moved or renamed - likely
- Possible move or rename - ambiguous

Compiler-derived, likely, inferred, unresolved, and property-change results must remain visually and textually distinct.

1.4 3. Observed repository attention

Tadori records observable repository interactions. It does not expose chain of thought or claim complete knowledge of the agent's context.

Observed event types:

- mcp_returned
- file_read_observed
- plan_mentioned
- modified
- test_selected
- test_executed
- capture_interrupted

The default state is **not observed inspected**. This means only that Tadori did not observe inspection through a supported MCP call, hook, imported transcript, or recognized tool event during that task.

Do not say:

- the agent ignored this;
- the agent did not know this;
- the agent never saw this.

Task-level observation coverage is one of:

- complete_for_registered_sources
- partial
- unknown

Package-level overlays must aggregate child events fractionally. A package with one observed child among two hundred is never painted as fully inspected.

1.5 4. Test linkage is not test coverage

Tadori distinguishes:

- statically linked test;
- naming-associated test;
- package-associated test;
- historically associated test;
- executed test;
- passed or failed test;
- runtime-covering test, unavailable in v1.

find_tests must say **Likely relevant tests**. Static linkage does not prove behavioral coverage.

1.6 5. Graph-quality validation

Precision and recall use different ground-truth methods.

1.6.1 Precision on real repositories

Use stratified random samples of emitted relations. Report two-sided 95% Wilson confidence intervals. Ambiguous reviewer judgments count as incorrect for release gating.

For a perfect sample, the Wilson lower bound is:

LB = 1 / (1 + z^2 / n), where z = 1.959964.

Therefore:

- LB >= 0.98 requires at least 189 perfect observations; use 200.
- LB >= 0.99 requires at least 381 perfect observations; use 400.

1.6.2 Corrected precision and fixture-recall table

Relation	Real audit n	Required 95% Wilson lower bound	Maximum accepted errors	Fixture recall gate
Imports	200	>= 0.98	0	>= 0.98
Exports/re-exports	200	>= 0.98	0	>= 0.98
Definitions	400	>= 0.99	0	>= 0.99
References	200	>= 0.97	1	>= 0.95
Resolved calls	200	>= 0.90	11	>= 0.80
Heuristic calls	200	>= 0.70	47	no global recall claim

Relation	Real audit n	Required 95% Wilson lower bound	Maximum accepted errors	Fixture recall gate
Express routes	200	>= 0.95	3	>= 0.90
Next.js routes	200	>= 0.98	0	>= 0.95
Compiler-linked tests	200	>= 0.90	11	>= 0.85
Naming-associated tests	150	>= 0.75	27	no release recall gate
Exact ADR path links	200	>= 0.95	3	>= 0.90
Unique-symbol ADR links	200	>= 0.90	11	>= 0.80

If a stratum population is smaller than its target sample, audit the entire population as a census. High-bar relations with any false positive fail that census.

1.6.3 Recall

Recall is claimed only on synthetic or manually constructed fixtures whose complete expected graph is known by construction. Do not publish real-repository recall numbers without a complete audit.

1.6.4 Required relation metrics

Report separately:

- precision;
- fixture recall;
- F1 where both exist;
- unresolved candidate rate;
- excluded dynamic call count.

1.7 6. Multi-resolution visualization

Do not render one all-symbol graph.

Three levels:

1. Package aggregate - always resident.
2. File subgraph - loaded per selected package or task region.
3. Symbol task region - loaded only around anchors, direct relations, linked tests, paths, and bounded impact regions.

Positions are cached per level. Existing unrelated regions do not move during ordinary reindexing. New entities use local placement and bounded relaxation.

Cross-level edges terminate at aggregate portals until expanded. Aggregated counts must reconcile with the underlying edges.

The default interface remains stable 2D. Optional 2.5D binds depth to exactly one queryable field. Experimental 3D remains off by default and uses the same graph queries.

Anti-decoration rule:

> Every visual channel must map to a named stored or computed field and must expose the underlying rows.

1.8 7. Evaluation sequence

1.8.1 During the 12-week build

Evaluate engineering properties without external participants:

- extraction quality;
- fixture recall;
- sampled precision;
- retrieval quality;
- task success;
- unrelated modifications;
- missed consumers;
- stale-result behavior;
- incremental indexing;
- extracted graph-diff correctness.

1.8.2 Post-build pilot

Use 12-16 participants to estimate effect sizes, variance, usability failures, disorientation, and task quality. This pilot is not a significance test.

Navigation and tracing tasks may be within-subject using different repos and counterbalancing. Patch review must be between-subject because seeing the same defect twice contaminates the comparison.

1.8.3 Formal study

Use pilot variance and power analysis. Provisional targets:

- approximately 40 recruits for within-subject navigation/tracing;
- 48-64 participants for repeated unique patch-review tasks analyzed with mixed effects.

1.8.4 Minimum useful depth effects

A depth-based condition must produce at least one of:

- = 8 percentage-point accuracy improvement with <= 10% time increase and <= 5 NASA-TLX point increase;
- = 10% time reduction while remaining within a -3 percentage-point accuracy noninferiority margin;
- = 10 percentage-point defect-detection improvement without increasing false rejection by more than 5 points.

Free-orbit 3D is removed if it is materially slower, less accurate, more cognitively demanding, disorienting, frequently abandoned for 2D, or requires bespoke semantics outside the common query model.

1.9 8. Privacy and local operation

Default bind address: 127.0.0.1.

Apply the union of:

- .gitignore
- .tadoriignore
- built-in generated, binary, and large-file exclusions

Default exclusions include node_modules, build output, coverage, caches, maps, minified JavaScript, binaries, and large generated artifacts.

Redaction families include:

- AWS access keys;
- GitHub tokens;
- Google API keys;
- Slack tokens;
- generic sk- keys;
- JWTs;
- private-key blocks;
- .env values whose keys indicate secrets, passwords, tokens, API keys, private keys, database URLs, or connection strings.

Support:

- source-body suppression;
- excerpt-size limits;
- separate retention for snapshots, retrieval events, hook events, plans, test output, and excerpts;
- explicit repository configuration and consent;
- purge commands for tasks, logs, source bodies, snapshots, and complete repositories.

1.10 9. Executable SQLite schema

The implementation uses ordered migrations. The schema below is the accepted structural contract; application code calculates canonical hashes.

1.10.1 Migration 001 - repositories, snapshots, stable entities, memberships, evidence

```
PRAGMA foreign_keys = ON;
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;

CREATE TABLE IF NOT EXISTS schema_migrations (
   version INTEGER PRIMARY KEY,
   applied_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

BEGIN IMMEDIATE;

CREATE TABLE repositories (
   id INTEGER PRIMARY KEY,
   root_path TEXT NOT NULL UNIQUE,
   created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE repository_snapshots (
   id INTEGER PRIMARY KEY,
   repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
   kind TEXT NOT NULL CHECK (kind IN ('commit','working_tree','staged','patch')),
   label TEXT,
   base_commit_sha TEXT,
   workspace_hash TEXT NOT NULL,
   parent_snapshot_id INTEGER REFERENCES repository_snapshots(id) ON DELETE SET
NULL,
   created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
   pinned INTEGER NOT NULL DEFAULT 0 CHECK (pinned IN (0,1)),
   status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active','pruned')),
   UNIQUE (repo_id, kind, workspace_hash)
);

CREATE INDEX idx_snapshots_repo_created ON repository_snapshots(repo_id,
created_at);

CREATE TABLE file_entities (
   id INTEGER PRIMARY KEY,
   repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
```

```
  file_key TEXT NOT NULL,
  origin_identity TEXT NOT NULL,
  collision_index INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (repo_id, file_key),
  UNIQUE (repo_id, origin_identity)
);

CREATE TABLE snapshot_files (
  snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE
CASCADE,
  file_id INTEGER NOT NULL REFERENCES file_entities(id) ON DELETE RESTRICT,
  path TEXT NOT NULL,
  normalized_path TEXT NOT NULL,
  package_name TEXT,
  language TEXT,
  content_hash TEXT NOT NULL,
  size_bytes INTEGER NOT NULL DEFAULT 0,
  is_generated INTEGER NOT NULL DEFAULT 0 CHECK (is_generated IN (0,1)),
  is_binary INTEGER NOT NULL DEFAULT 0 CHECK (is_binary IN (0,1)),
  PRIMARY KEY (snapshot_id, file_id),
  UNIQUE (snapshot_id, normalized_path)
);

CREATE INDEX idx_snapshot_files_path ON snapshot_files(snapshot_id,
normalized_path);
CREATE INDEX idx_snapshot_files_package ON snapshot_files(snapshot_id,
package_name);

CREATE TABLE node_entities (
  id INTEGER PRIMARY KEY,
  repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
  entity_key TEXT NOT NULL,
  canonical_identity TEXT NOT NULL,
  collision_index INTEGER NOT NULL DEFAULT 0,
  kind TEXT NOT NULL CHECK (kind IN (
    'package','file','function','method','class','interface','type','route','test',
    'adr','doc_section','external_dep','unresolved'
  )),
  qualified_name TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (repo_id, entity_key),
  UNIQUE (repo_id, canonical_identity)
```

```
);

CREATE INDEX idx_node_entities_name ON node_entities(repo_id, qualified_name);
CREATE INDEX idx_node_entities_kind ON node_entities(repo_id, kind);

CREATE TABLE snapshot_nodes (
   snapshot_id INTEGER NOT NULL,
   node_id INTEGER NOT NULL REFERENCES node_entities(id) ON DELETE RESTRICT,
   file_id INTEGER,
   display_name TEXT NOT NULL,
   span_start INTEGER,
   span_end INTEGER,
   line_start INTEGER,
   line_end INTEGER,
   signature TEXT,
   body_hash TEXT,
   exported INTEGER NOT NULL DEFAULT 0 CHECK (exported IN (0,1)),
   analyzer_version TEXT NOT NULL,
   PRIMARY KEY (snapshot_id, node_id),
   FOREIGN KEY (snapshot_id) REFERENCES repository_snapshots(id) ON DELETE
CASCADE,
   FOREIGN KEY (snapshot_id, file_id) REFERENCES snapshot_files(snapshot_id, file_id)
ON DELETE CASCADE,
   CHECK ((span_start IS NULL AND span_end IS NULL) OR
        (span_start IS NOT NULL AND span_end IS NOT NULL AND span_end >=
span_start))
);

CREATE INDEX idx_snapshot_nodes_file ON snapshot_nodes(snapshot_id, file_id);
CREATE INDEX idx_snapshot_nodes_lines ON snapshot_nodes(snapshot_id, file_id,
line_start, line_end);

CREATE TABLE edge_entities (
   id INTEGER PRIMARY KEY,
   repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
   entity_key TEXT NOT NULL,
   canonical_identity TEXT NOT NULL,
   collision_index INTEGER NOT NULL DEFAULT 0,
   src_node_id INTEGER NOT NULL REFERENCES node_entities(id) ON DELETE
RESTRICT,
   dst_node_id INTEGER NOT NULL REFERENCES node_entities(id) ON DELETE
RESTRICT,
   relation TEXT NOT NULL CHECK (relation IN (
```

```
      'contains','imports','exports','references','calls','implements','extends',
      'tests','routes_to','documents','changed_with'
    )),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (repo_id, entity_key),
    UNIQUE (repo_id, canonical_identity)
);

CREATE INDEX idx_edge_entities_src ON edge_entities(src_node_id, relation);
CREATE INDEX idx_edge_entities_dst ON edge_entities(dst_node_id, relation);

CREATE TABLE snapshot_edges (
    snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE
CASCADE,
    edge_id INTEGER NOT NULL REFERENCES edge_entities(id) ON DELETE RESTRICT,
    origin TEXT NOT NULL CHECK (origin IN
('compiler','heuristic','git','doc','human','llm')),
    confidence TEXT NOT NULL CHECK (confidence IN ('certain','likely','inferred')),
    resolution TEXT NOT NULL CHECK (resolution IN ('resolved','partial','unresolved')),
    analyzer_version TEXT NOT NULL,
    PRIMARY KEY (snapshot_id, edge_id)
);

CREATE INDEX idx_snapshot_edges_snapshot ON snapshot_edges(snapshot_id, edge_id);
CREATE INDEX idx_snapshot_edges_origin ON snapshot_edges(snapshot_id, origin,
confidence, resolution);

CREATE TABLE evidence_items (
    id INTEGER PRIMARY KEY,
    snapshot_id INTEGER NOT NULL,
    file_id INTEGER NOT NULL,
    evidence_kind TEXT NOT NULL CHECK (evidence_kind IN (
      'source','documentation','git','human_annotation','tool_event'
    )),
    line_start INTEGER,
    line_end INTEGER,
    column_start INTEGER,
    column_end INTEGER,
    commit_sha TEXT,
    excerpt_hash TEXT,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (snapshot_id, file_id) REFERENCES snapshot_files(snapshot_id, file_id)
ON DELETE CASCADE,
```

```
    CHECK ((line_start IS NULL AND line_end IS NULL) OR
           (line_start IS NOT NULL AND line_end IS NOT NULL AND line_end >= line_start))
);

CREATE TABLE node_evidence (
    snapshot_id INTEGER NOT NULL,
    node_id INTEGER NOT NULL,
    evidence_id INTEGER NOT NULL REFERENCES evidence_items(id) ON DELETE
CASCADE,
    PRIMARY KEY (snapshot_id, node_id, evidence_id),
    FOREIGN KEY (snapshot_id, node_id) REFERENCES snapshot_nodes(snapshot_id,
node_id) ON DELETE CASCADE
);

CREATE TABLE edge_evidence (
    snapshot_id INTEGER NOT NULL,
    edge_id INTEGER NOT NULL,
    evidence_id INTEGER NOT NULL REFERENCES evidence_items(id) ON DELETE
CASCADE,
    PRIMARY KEY (snapshot_id, edge_id, evidence_id),
    FOREIGN KEY (snapshot_id, edge_id) REFERENCES snapshot_edges(snapshot_id,
edge_id) ON DELETE CASCADE
);

INSERT INTO schema_migrations(version) VALUES (1);
COMMIT;
```

1.10.2 Migration 002 - boundaries and explicit decisions

```
PRAGMA foreign_keys = ON;
BEGIN IMMEDIATE;

CREATE TABLE boundary_entities (
    id INTEGER PRIMARY KEY,
    repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
    boundary_key TEXT NOT NULL,
    name TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (repo_id, boundary_key),
    UNIQUE (repo_id, name)
);

CREATE TABLE snapshot_boundaries (
    snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE
```

```
CASCADE,
    boundary_id INTEGER NOT NULL REFERENCES boundary_entities(id) ON DELETE
RESTRICT,
    rule_kind TEXT NOT NULL CHECK (rule_kind IN ('allow','deny','layer','ownership')),
    rule_json TEXT NOT NULL CHECK (json_valid(rule_json)),
    origin TEXT NOT NULL CHECK (origin IN ('config','documentation','human')),
    confidence TEXT NOT NULL CHECK (confidence IN ('certain','likely')),
    source_file_id INTEGER,
    PRIMARY KEY (snapshot_id, boundary_id),
    FOREIGN KEY (snapshot_id, source_file_id) REFERENCES snapshot_files(snapshot_id,
file_id) ON DELETE SET NULL
);

CREATE TABLE boundary_evidence (
    snapshot_id INTEGER NOT NULL,
    boundary_id INTEGER NOT NULL,
    evidence_id INTEGER NOT NULL REFERENCES evidence_items(id) ON DELETE
CASCADE,
    PRIMARY KEY (snapshot_id, boundary_id, evidence_id),
    FOREIGN KEY (snapshot_id, boundary_id) REFERENCES
snapshot_boundaries(snapshot_id, boundary_id) ON DELETE CASCADE
);

CREATE TABLE decision_entities (
    id INTEGER PRIMARY KEY,
    repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
    decision_key TEXT NOT NULL,
    kind TEXT NOT NULL CHECK (kind IN ('adr','doc','annotation')),
    title TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (repo_id, decision_key)
);

CREATE TABLE snapshot_decisions (
    snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE
CASCADE,
    decision_id INTEGER NOT NULL REFERENCES decision_entities(id) ON DELETE
RESTRICT,
    status TEXT NOT NULL CHECK (status IN
('proposed','accepted','deprecated','superseded','unknown')),
    source_file_id INTEGER,
    body_excerpt TEXT,
    content_hash TEXT,
```

```
  PRIMARY KEY (snapshot_id, decision_id),
  FOREIGN KEY (snapshot_id, source_file_id) REFERENCES snapshot_files(snapshot_id,
file_id) ON DELETE SET NULL
);

CREATE TABLE decision_evidence (
  snapshot_id INTEGER NOT NULL,
  decision_id INTEGER NOT NULL,
  evidence_id INTEGER NOT NULL REFERENCES evidence_items(id) ON DELETE
CASCADE,
  PRIMARY KEY (snapshot_id, decision_id, evidence_id),
  FOREIGN KEY (snapshot_id, decision_id) REFERENCES
snapshot_decisions(snapshot_id, decision_id) ON DELETE CASCADE
);

CREATE TABLE decision_links (
  id INTEGER PRIMARY KEY,
  snapshot_id INTEGER NOT NULL,
  decision_id INTEGER NOT NULL,
  node_id INTEGER,
  edge_id INTEGER,
  boundary_id INTEGER,
  confidence TEXT NOT NULL CHECK (confidence IN ('certain','likely')),
  evidence_id INTEGER NOT NULL REFERENCES evidence_items(id) ON DELETE
CASCADE,
  FOREIGN KEY (snapshot_id, decision_id) REFERENCES
snapshot_decisions(snapshot_id, decision_id) ON DELETE CASCADE,
  FOREIGN KEY (snapshot_id, node_id) REFERENCES snapshot_nodes(snapshot_id,
node_id) ON DELETE CASCADE,
  FOREIGN KEY (snapshot_id, edge_id) REFERENCES snapshot_edges(snapshot_id,
edge_id) ON DELETE CASCADE,
  FOREIGN KEY (snapshot_id, boundary_id) REFERENCES
snapshot_boundaries(snapshot_id, boundary_id) ON DELETE CASCADE,
  CHECK ((node_id IS NOT NULL) + (edge_id IS NOT NULL) + (boundary_id IS NOT
NULL) = 1)
);

CREATE UNIQUE INDEX idx_decision_link_node ON decision_links(snapshot_id,
decision_id, node_id) WHERE node_id IS NOT NULL;
CREATE UNIQUE INDEX idx_decision_link_edge ON decision_links(snapshot_id,
decision_id, edge_id) WHERE edge_id IS NOT NULL;
CREATE UNIQUE INDEX idx_decision_link_boundary ON decision_links(snapshot_id,
decision_id, boundary_id) WHERE boundary_id IS NOT NULL;
```

```
INSERT INTO schema_migrations(version) VALUES (2);
COMMIT;
```

1.10.3 Migration 003 - tasks, retrieval, observed events, changes, tests

```
PRAGMA foreign_keys = ON;
BEGIN IMMEDIATE;

CREATE TABLE tasks (
  id INTEGER PRIMARY KEY,
  repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
  base_snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON
DELETE RESTRICT,
  agent TEXT NOT NULL,
  description TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN
('active','completed','aborted','deleted')),
  observation_coverage TEXT NOT NULL DEFAULT 'partial' CHECK (
    observation_coverage IN ('complete_for_registered_sources','partial','unknown')
  ),
  started_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ended_at TEXT
);

CREATE INDEX idx_tasks_repo_started ON tasks(repo_id, started_at);

CREATE TABLE retrieval_events (
  id INTEGER PRIMARY KEY,
  task_id INTEGER NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE
RESTRICT,
  tool TEXT NOT NULL CHECK (tool IN (
    'repo_overview','find_symbol','symbol_context','find_tests','impact','path'
  )),
  args_json TEXT NOT NULL CHECK (json_valid(args_json)),
  requested_token_budget INTEGER,
  estimated_response_tokens INTEGER,
  truncated INTEGER NOT NULL DEFAULT 0 CHECK (truncated IN (0,1)),
  next_cursor TEXT,
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_retrieval_events_task ON retrieval_events(task_id, created_at);
```

```
CREATE TABLE retrieval_result_nodes (
    event_id INTEGER NOT NULL REFERENCES retrieval_events(id) ON DELETE
CASCADE,
    node_id INTEGER NOT NULL REFERENCES node_entities(id) ON DELETE RESTRICT,
    rank_position INTEGER NOT NULL,
    score REAL,
    representation TEXT NOT NULL CHECK (representation IN
('body','signature','name','aggregate')),
    stale INTEGER NOT NULL DEFAULT 0 CHECK (stale IN (0,1)),
    PRIMARY KEY (event_id, node_id)
);

CREATE TABLE retrieval_result_edges (
    event_id INTEGER NOT NULL REFERENCES retrieval_events(id) ON DELETE
CASCADE,
    edge_id INTEGER NOT NULL REFERENCES edge_entities(id) ON DELETE RESTRICT,
    rank_position INTEGER NOT NULL,
    score REAL,
    stale INTEGER NOT NULL DEFAULT 0 CHECK (stale IN (0,1)),
    PRIMARY KEY (event_id, edge_id)
);

CREATE TABLE retrieval_omissions (
    id INTEGER PRIMARY KEY,
    event_id INTEGER NOT NULL REFERENCES retrieval_events(id) ON DELETE
CASCADE,
    target_kind TEXT NOT NULL CHECK (target_kind IN ('node','edge')),
    node_id INTEGER REFERENCES node_entities(id) ON DELETE RESTRICT,
    edge_id INTEGER REFERENCES edge_entities(id) ON DELETE RESTRICT,
    rank_position INTEGER NOT NULL,
    score REAL,
    reason TEXT NOT NULL,
    CHECK ((target_kind = 'node' AND node_id IS NOT NULL AND edge_id IS NULL) OR
           (target_kind = 'edge' AND edge_id IS NOT NULL AND node_id IS NULL))
);

CREATE TABLE agent_events (
    id INTEGER PRIMARY KEY,
    task_id INTEGER NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
    snapshot_id INTEGER REFERENCES repository_snapshots(id) ON DELETE RESTRICT,
    event_type TEXT NOT NULL CHECK (event_type IN (

'file_read_observed','plan_mentioned','modified','test_selected','test_executed','capture_int
```

```
errupted'
  )),
  source TEXT NOT NULL CHECK (source IN
('claude_hook','codex_log','transcript','manual')),
  payload_json TEXT CHECK (payload_json IS NULL OR json_valid(payload_json)),
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE agent_event_targets (
  id INTEGER PRIMARY KEY,
  event_id INTEGER NOT NULL REFERENCES agent_events(id) ON DELETE CASCADE,
  target_kind TEXT NOT NULL CHECK (target_kind IN ('file','node')),
  file_id INTEGER REFERENCES file_entities(id) ON DELETE RESTRICT,
  node_id INTEGER REFERENCES node_entities(id) ON DELETE RESTRICT,
  CHECK ((target_kind = 'file' AND file_id IS NOT NULL AND node_id IS NULL) OR
      (target_kind = 'node' AND node_id IS NOT NULL AND file_id IS NULL))
);

CREATE UNIQUE INDEX idx_agent_event_target_file
  ON agent_event_targets(event_id, file_id)
  WHERE target_kind = 'file' AND file_id IS NOT NULL;

CREATE UNIQUE INDEX idx_agent_event_target_node
  ON agent_event_targets(event_id, node_id)
  WHERE target_kind = 'node' AND node_id IS NOT NULL;

CREATE TABLE change_sets (
  id INTEGER PRIMARY KEY,
  task_id INTEGER NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  base_snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON
DELETE RESTRICT,
  head_snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON
DELETE RESTRICT,
  kind TEXT NOT NULL CHECK (kind IN ('working_tree','staged','commit','patch')),
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK (base_snapshot_id <> head_snapshot_id)
);

CREATE TABLE change_set_files (
  change_set_id INTEGER NOT NULL REFERENCES change_sets(id) ON DELETE
CASCADE,
  file_id INTEGER NOT NULL REFERENCES file_entities(id) ON DELETE RESTRICT,
  change_kind TEXT NOT NULL CHECK (change_kind IN
```

```
('added','modified','deleted','renamed')),
    before_hash TEXT,
    after_hash TEXT,
    old_path TEXT,
    new_path TEXT,
    PRIMARY KEY (change_set_id, file_id)
);

CREATE TABLE test_runs (
    id INTEGER PRIMARY KEY,
    task_id INTEGER NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
    snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE
RESTRICT,
    command_redacted TEXT NOT NULL,
    status TEXT NOT NULL CHECK (status IN
('running','passed','failed','cancelled','unknown')),
    exit_code INTEGER,
    started_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ended_at TEXT,
    output_retained INTEGER NOT NULL DEFAULT 0 CHECK (output_retained IN (0,1))
);

CREATE TABLE test_run_cases (
    id INTEGER PRIMARY KEY,
    test_run_id INTEGER NOT NULL REFERENCES test_runs(id) ON DELETE CASCADE,
    test_node_id INTEGER REFERENCES node_entities(id) ON DELETE SET NULL,
    test_name TEXT NOT NULL,
    status TEXT NOT NULL CHECK (status IN ('passed','failed','skipped','unknown')),
    duration_ms INTEGER
);

INSERT INTO schema_migrations(version) VALUES (3);
COMMIT;
```

1.10.4 Migration 004 - layouts and quarantined summaries

```
PRAGMA foreign_keys = ON;
BEGIN IMMEDIATE;

CREATE TABLE layout_positions (
    repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
    abstraction_level TEXT NOT NULL CHECK (abstraction_level IN
('package','file','symbol')),
    view_key TEXT NOT NULL DEFAULT 'base',
```

```
    node_id INTEGER NOT NULL REFERENCES node_entities(id) ON DELETE CASCADE,
    x REAL NOT NULL,
    y REAL NOT NULL,
    z REAL NOT NULL DEFAULT 0,
    pinned INTEGER NOT NULL DEFAULT 0 CHECK (pinned IN (0,1)),
    anchor_group TEXT,
    layout_version INTEGER NOT NULL DEFAULT 1,
    last_snapshot_id INTEGER REFERENCES repository_snapshots(id) ON DELETE SET
NULL,
    updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (repo_id, abstraction_level, view_key, node_id)
);

CREATE INDEX idx_layout_level ON layout_positions(repo_id, abstraction_level,
view_key);

CREATE TABLE summaries (
    id INTEGER PRIMARY KEY,
    snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE
CASCADE,
    node_id INTEGER NOT NULL REFERENCES node_entities(id) ON DELETE CASCADE,
    model TEXT NOT NULL,
    prompt_hash TEXT NOT NULL,
    summary_text TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (snapshot_id, node_id, model, prompt_hash)
);

INSERT INTO schema_migrations(version) VALUES (4);
COMMIT;
```

1.10.5 Migration 005 - full-text search

```
PRAGMA foreign_keys = ON;
BEGIN IMMEDIATE;

CREATE VIRTUAL TABLE node_fts USING fts5(
    snapshot_id UNINDEXED,
    node_id UNINDEXED,
    display_name,
    qualified_name,
    signature,
    path,
    tokenize = 'unicode61'
```

```
);

INSERT INTO schema_migrations(version) VALUES (5);
COMMIT;
```

1.11 10. Required snapshot validation

The schema does not enforce that each snapshot edge's endpoint nodes are members of the same snapshot. Run this after every completed snapshot build:

```
SELECT
  se.snapshot_id,
  se.edge_id,
  ee.src_node_id,
  ee.dst_node_id,
  CASE WHEN src.node_id IS NULL THEN 1 ELSE 0 END AS
missing_source_membership,
  CASE WHEN dst.node_id IS NULL THEN 1 ELSE 0 END AS
missing_destination_membership
FROM snapshot_edges AS se
JOIN edge_entities AS ee ON ee.id = se.edge_id
LEFT JOIN snapshot_nodes AS src
 ON src.snapshot_id = se.snapshot_id AND src.node_id = ee.src_node_id
LEFT JOIN snapshot_nodes AS dst
 ON dst.snapshot_id = se.snapshot_id AND dst.node_id = ee.dst_node_id
WHERE src.node_id IS NULL OR dst.node_id IS NULL;
```

A valid snapshot returns zero rows. Invalid snapshots are not served through MCP and do not replace the last valid active snapshot.

1.12 11. Three-way extracted-edge diff

A complete graph diff includes added edges, removed edges, and membership-property changes on an edge that exists in both snapshots.

```
WITH edge_diff AS (
  SELECT
    'added' AS change_kind,
    head.edge_id,
    NULL AS before_origin,
    NULL AS before_confidence,
    NULL AS before_resolution,
    head.origin AS after_origin,
    head.confidence AS after_confidence,
    head.resolution AS after_resolution
  FROM snapshot_edges AS head
```

```
WHERE head.snapshot_id = :head_snapshot
  AND NOT EXISTS (
      SELECT 1 FROM snapshot_edges AS base
      WHERE base.snapshot_id = :base_snapshot
        AND base.edge_id = head.edge_id
  )

UNION ALL

SELECT
    'removed',
    base.edge_id,
    base.origin,
    base.confidence,
    base.resolution,
    NULL,
    NULL,
    NULL
FROM snapshot_edges AS base
WHERE base.snapshot_id = :base_snapshot
  AND NOT EXISTS (
      SELECT 1 FROM snapshot_edges AS head
      WHERE head.snapshot_id = :head_snapshot
        AND head.edge_id = base.edge_id
  )

UNION ALL

SELECT
    'resolution_or_provenance_changed',
    head.edge_id,
    base.origin,
    base.confidence,
    base.resolution,
    head.origin,
    head.confidence,
    head.resolution
FROM snapshot_edges AS base
JOIN snapshot_edges AS head
  ON head.edge_id = base.edge_id
 AND head.snapshot_id = :head_snapshot
WHERE base.snapshot_id = :base_snapshot
  AND (
```

```
        base.origin <> head.origin OR
        base.confidence <> head.confidence OR
        base.resolution <> head.resolution
    )
)
SELECT
  d.change_kind,
  src.qualified_name AS source,
  e.relation,
  dst.qualified_name AS destination,
  d.before_origin,
  d.before_confidence,
  d.before_resolution,
  d.after_origin,
  d.after_confidence,
  d.after_resolution
FROM edge_diff AS d
JOIN edge_entities AS e ON e.id = d.edge_id
JOIN node_entities AS src ON src.id = e.src_node_id
JOIN node_entities AS dst ON dst.id = e.dst_node_id;
```

1.13 12. Rename and move coalescing

Raw entity identity remains unchanged. Review mode adds a presentation-time coalescing pass.

1.13.1 Stage A - likely move with name preserved

Pair one removed node with one added node when all match:

- kind;
- unqualified name;
- body hash;
- analyzer version.

1.13.2 Stage B - likely symbol rename

For unmatched nodes, pair one removed node with one added node when all match:

- kind;
- body hash;
- analyzer version;
- unqualified names differ.

Recursive functions and symbols whose bodies contain self-references may not Stage-B match after rename because the body text changes with the name. The safe fallback is the raw added/removed diff.

1.13.3 Ambiguity

If multiple removed or added candidates share a candidate signature and no unique Git rename tie-breaker exists, do not coalesce automatically. Show **Possible move or rename - ambiguous** and preserve the raw rows.

1.13.4 Edges

After node pairs exist, pair removed and added edges only when:

- relation is identical;
- mapped source and destination endpoints match;
- exactly one corresponding added edge exists.

Show both raw and coalesced counts.

1.14 13. Foreign-key-safe garbage collection

Entity collection must respect all surviving ON DELETE RESTRICT references.

```
PRAGMA foreign_keys = ON;
BEGIN IMMEDIATE;

DELETE FROM edge_entities
WHERE NOT EXISTS (SELECT 1 FROM snapshot_edges WHERE snapshot_edges.edge_id
= edge_entities.id)
  AND NOT EXISTS (SELECT 1 FROM retrieval_result_edges WHERE
retrieval_result_edges.edge_id = edge_entities.id)
  AND NOT EXISTS (SELECT 1 FROM retrieval_omissions WHERE
retrieval_omissions.edge_id = edge_entities.id)
  AND NOT EXISTS (SELECT 1 FROM decision_links WHERE decision_links.edge_id =
edge_entities.id);

DELETE FROM node_entities
WHERE NOT EXISTS (SELECT 1 FROM snapshot_nodes WHERE
snapshot_nodes.node_id = node_entities.id)
  AND NOT EXISTS (SELECT 1 FROM retrieval_result_nodes WHERE
retrieval_result_nodes.node_id = node_entities.id)
  AND NOT EXISTS (SELECT 1 FROM retrieval_omissions WHERE
retrieval_omissions.node_id = node_entities.id)
  AND NOT EXISTS (SELECT 1 FROM agent_event_targets WHERE
agent_event_targets.node_id = node_entities.id)
  AND NOT EXISTS (SELECT 1 FROM decision_links WHERE decision_links.node_id =
```

```
node_entities.id)
  AND NOT EXISTS (
    SELECT 1 FROM edge_entities
    WHERE edge_entities.src_node_id = node_entities.id
      OR edge_entities.dst_node_id = node_entities.id
  );

DELETE FROM file_entities
WHERE NOT EXISTS (SELECT 1 FROM snapshot_files WHERE snapshot_files.file_id =
file_entities.id)
  AND NOT EXISTS (SELECT 1 FROM agent_event_targets WHERE
agent_event_targets.file_id = file_entities.id)
  AND NOT EXISTS (SELECT 1 FROM change_set_files WHERE change_set_files.file_id =
file_entities.id);

DELETE FROM boundary_entities
WHERE NOT EXISTS (SELECT 1 FROM snapshot_boundaries WHERE
snapshot_boundaries.boundary_id = boundary_entities.id)
  AND NOT EXISTS (SELECT 1 FROM decision_links WHERE decision_links.boundary_id
= boundary_entities.id);

DELETE FROM decision_entities
WHERE NOT EXISTS (SELECT 1 FROM snapshot_decisions WHERE
snapshot_decisions.decision_id = decision_entities.id);

COMMIT;
PRAGMA foreign_key_check;
```

1.15 14. Updated twelve-week gates

1.15.1 Weeks 1-2

- migrations run on an empty SQLite database;
- commit and working-tree snapshots coexist;
- foreign_key_check returns zero rows;
- dangling endpoint memberships equal zero;
- 150k LOC initial index under 60 seconds on the target machine;
- import/export fixture gates pass.

1.15.2 Week 3

- relation-specific fixture recall reported;
- real-repository precision samples collected;
- unresolved call rate reported;
- unsupported relations disabled rather than mislabeled.

1.15.3 Week 4

- exactly six MCP tools;
- every call logged;
- hook interruption changes observation coverage to partial.

1.15.4 Week 5

- hard includes and omission manifests;
- 100% of truncated responses include an omission manifest;
- no direct compiler-certain caller/callee is silently dropped.

1.15.5 Week 6

Latency gates:

- single-file change: under 2 seconds p95;
- package-scope invalidation: under 10 seconds p95;
- dependency-invalidated region: best effort with measured, documented ceiling and stale flags retained until completion.

1.15.6 Weeks 7-8

- package map interactive in under 5 seconds for 250k LOC;
- file expansion does not move unrelated packages;
- symbol expansion does not load the full symbol graph;
- aggregate coverage reconciles with child observations.

1.15.7 Week 9

- working-tree comparison works without a commit;
- fixture diff recall and sampled real-diff precision meet relation targets;
- UI distinguishes extracted graph differences from runtime architecture truth.

1.15.8 Week 10

- 2D, 2.5D, and 3D consume the same entity and overlay query results;
- every depth field maps to a named field;
- this remains the first work cut if the schedule slips.

1.15.9 Week 11

Run the agent benchmark against plain Claude Code and a real existing graph-MCP baseline. Report results regardless of outcome.

1.15.10 Week 12

Do not attempt the full human study. Freeze the prototype, run an internal visual-task dry run, finalize the pilot protocol, verify privacy behavior, publish benchmark results, and record the demo.

1.16 15. Failure conditions

- If graph retrieval lowers task success or produces no improvement in unrelated edits, missed consumers, unsupported claims, or evidence quality, remove it as a product claim.
- If reviewers ignore the attention overlay or it creates overconfidence, retain a table and diff but drop the map.
- If decision links remain inaccurate or unused, reduce them to a simple documentation-reference panel.
- If 3D misses the predefined useful-effect criteria, delete it from the product branch and report the negative result.
- Stop entirely if graph quality remains untrustworthy or interactive indexing/stale-state handling cannot be made reliable.

1.17 16. Frozen-spec note

Tadori v2.1 was accepted after independent execution checks confirmed:

- migrations build on an empty SQLite database;
- the worked snapshot example inserts cleanly;
- the three-way diff returns added, removed, and resolution/provenance-change rows;
- dangling endpoint validation returns zero;
- foreign_key_check returns zero rows;
- pipe-delimited canonical SHA-256 identities match;
- Wilson table calculations are attainable and consistent.

No further specification revision should occur without a new defect report.